JAMHURI YA MUUNGANO WA TANZANIA · E-GOVERNMENT AUTHORITY (eGA)

TECHNICAL REFERENCE AND ARCHITECTURE GUIDE

eSiri

How the proof of concept is built: architecture, agent loop, safety model, data flows and code

Prepared by Joseph Laurent
Institution e-Government Authority (eGA), Tanzania
Date September 2026
Scope The working proof of concept as built, operating a replica of e-Mikutano
Source of truth The code, as mapped in docs/CODEMAP.md of the eSIRI repository

How to use this document

This guide explains, in engineering depth, how the eSiri proof of concept works. It describes only what is built and running today. Every claim comes from the code map produced from the repository, and file and function names are given so you can open the exact place in the code.

Each chapter opens with an **"In plain words"** box. Read those boxes alone for a fast overview, then read the chapter body when you need the detail. For the non-technical version, see the companion **Plain-Language Guide**.

Contents

1. System overview
2. Architecture
3. Technology stack
4. How the agent knows what it can do
5. The agent loop in detail
6. Lifecycle of one real request
7. The hands: driver and ghost cursor
8. Safety model
9. The roles of OpenAI and Google, in depth
10. Speech layer
11. Bilingual support
12. State and persistence
13. Testing
14. Engineering decisions and bugs fixed
15. Known limitations
16. Reusing the pattern in other government systems
17. Resuming work on the project
18. Glossary

1. System overview

> **IN PLAIN WORDS**
>
> eSiri is an AI agent. You ask it for something in English or Swahili; it looks at the screen, decides the next step with an AI model, performs the step with a visible cursor, asks you before anything important, and records the whole task. In this proof of concept it operates a realistic replica of e-Mikutano that runs entirely in the browser.

The proof of concept consists of three parts:

- **The e-Mikutano mockup**: a React application imitating the e-Mikutano (Greenlight / BigBlueButton based) interface: landing page, rooms dashboard, room page with Recordings, Presentation, Access and Settings tabs, meeting screen, recordings list and audit log. It is fully usable by a human. Every actionable element carries `data-esiri-*` attributes that let the agent identify it.
- **The eSiri overlay** in the same browser tab: orb, side panel, speech, the agent loop controller, the snapshot builder, and the driver with its ghost cursor.
- **The backend**: a small FastAPI service that keeps conversation memory, assembles the prompt and calls OpenAI, synthesises Swahili speech and appends audit records to a file.

> **REMEMBER THIS**
>
> **The model decides; the code executes and enforces.** The model never touches the DOM or the application state. It only returns tool calls. All execution, validation and safety enforcement happen in browser code (`controller.ts`, `driver.ts`).

Key facts at a glance

Item	Value
Agent model	`gpt-4.1-mini` (OpenAI Chat Completions, tool calling, temperature 0.2)
Tools available to the model	7: `click`, `type_text`, `navigate`, `ask_confirmation`, `ask_user`, `set_language`, `finish`
Measured run: "Create a room called Board Meeting"	4 model calls, 15.0 s end to end, 4,156–4,493 prompt tokens and 20–71 completion tokens per call
System prompt size	14,603 characters (rules + app map + language line)
Languages	English and Swahili; 261 interface strings in each language
Safety limits per task	Sensitive clicks need an approved confirmation; max 30 tool executions; stop after 3 identical failures
Automated tests	27 Playwright tests (14 UI, 8 AI scenarios, 1 full demo, 4 visual), all passing
Ports	Frontend `localhost:5173` (Vite), backend `127.0.0.1:8000` (uvicorn)

2. Architecture

> **IN PLAIN WORDS**
>
> Three layers. The **browser** holds the mockup and all of eSiri's "hands and eyes". The **backend** on the same machine holds memory and talks to the AI. The **internet services** (OpenAI and Google) provide the thinking, the Swahili voice and speech recognition. Numbers on the arrows show what travels between them.

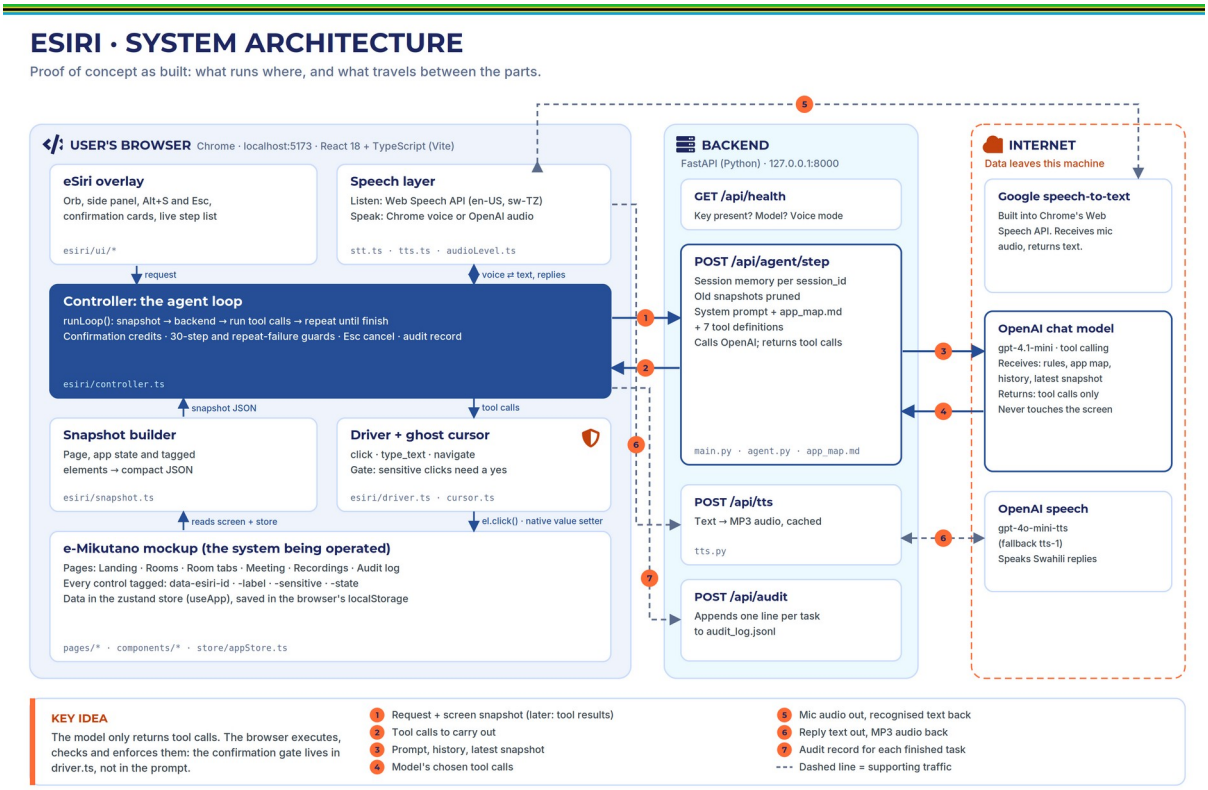


Figure 1. eSiri system architecture, as built. Numbered arrows are explained in the legend.

Components and responsibilities

Component	Main files	Responsibility
Mockup app	`pages/*`, `components/*`, `store/appStore.ts`	Human-usable imitation of e-Mikutano. Knows nothing about eSiri except the tags on its elements, built by `ez()` in `components/esiriProps.ts`.
eSiri overlay	`esiri/ui/EsiriRoot.tsx`, `Panel.tsx`, `Orb.tsx`, `esiri.css`	Orb, side panel, Alt+S and Esc shortcuts, wake mode, confirmation cards.
Controller	`esiri/controller.ts`	Owns each task: the loop, confirmation credits, guards, cancellation, audit record.
Snapshot builder	`esiri/snapshot.ts`	Turns the current screen and store into compact JSON for the model.
Driver and ghost cursor	`esiri/driver.ts`, `esiri/cursor.ts`	Performs click, type and navigate visibly; enforces the sensitive-click gate.

Component	Main files	Responsibility
Speech layer	`esiri/stt.ts`, `tts.ts`, `audioLevel.ts`	Listening, speaking, wake word, orb audio level.
Agent backend	`backend/app/main.py`, `agent.py`, `app_map.md`	Conversation memory per session, prompt assembly, OpenAI call, returns tool calls.
Speech backend	`backend/app/tts.py`	OpenAI text-to-speech with fallback model and in-memory cache.
Audit	`controller.ts finalizeTask()`, `appStore.ts addAudit`, `pages/Audit.tsx`, `main.py audit()`	One record per task, kept in the browser and appended to `backend/audit_log.jsonl`.

Runtime topology

- `./start.sh` checks for `python3` and `npm`, warns if `.env` or the key is missing, creates `backend/.venv` and installs dependencies on first run, runs `npm install` if needed, starts uvicorn on `127.0.0.1:8000` in the background, and runs the Vite dev server on port 5173 in the foreground. Ctrl+C stops both. Every path is quoted, so it works in folders with spaces.
- The browser calls `/api/*` on its own origin; the **Vite proxy** (`vite.config.ts`) forwards these calls to the backend, which avoids cross-origin problems. CORS additionally allows only `localhost:5173` and `127.0.0.1:5173`.
- The OpenAI key is read from `.env` by the backend only. The browser learns only whether a key exists (`/api/health` returns `has_key`).

Directory structure (abridged)

```
eSIRI/
├── start.sh                one-command install + run
├── .env.example            template for .env (real .env is git-ignored)
├── CLAUDE.md               briefing for future Claude Code sessions
├── docs/CODEMAP.md         detailed code map (source of truth for this guide)
├── backend/app/
│   ├── main.py             FastAPI endpoints, session memory, OpenAI call
│   ├── agent.py            SYSTEM_PROMPT, TOOLS, memory helpers (repair, trim, prune)
│   ├── app_map.md          the agent's knowledge base about the mockup
│   └── tts.py              OpenAI text-to-speech with fallback and cache
└── frontend/src/
    ├── App.tsx             routes, sign-in gate, layout shell, EsiriRoot
    ├── i18n/               en.ts, sw.ts (261 keys each), useT()
    ├── store/              appStore.ts (persisted state), data.ts (sample data)
    ├── components/         esiriProps.ts (ez), Header, Common (Modal, Toggle, Toasts)
    ├── pages/              Landing, Rooms, RoomPage, Recordings, Meeting, Audit
    └── esiri/              controller, driver, cursor, snapshot, api, stt, tts,
                            audioLevel, util, nav, esiriStore, ui/ (Root, Panel, Orb)
```

3. Technology stack

> **IN PLAIN WORDS**
>
> A React and TypeScript frontend (built with Vite) holds both the mockup and eSiri's client-side logic. A Python FastAPI backend holds memory and the AI calls. The browser's own speech features handle listening, OpenAI handles thinking and the Swahili voice, and Playwright tests everything end to end.

Frontend

Technology	Version	Role in eSiri	Why it was chosen
React	18.3.1	All UI: mockup and overlay.	Specified. Its controlled inputs are also why the "native setter" typing technique is needed (chapter 7).
TypeScript	5.6.3	Strict typing across the app.	Catches errors at build time; forces the Swahili dictionary to have exactly the English keys.
Vite	5.4.21	Dev server (port 5173), `/api` proxy, production build.	Fast; the proxy removes cross-origin issues in development.
react-router-dom	6.30.6	Page routes; `navigate` exposed to the agent through `esiri/nav.ts`.	Real URLs let the snapshot report the page and let the `navigate` tool work.
zustand	4.5.7	Two stores: `useApp` (persisted data and audit) and `useEsiri` (panel and status).	`getState()` lets non-React code (controller, driver, snapshot) read and write state directly; `persist` gives localStorage storage.
lucide-react	0.460.0	Icons.	Lightweight SVG icons, no brand logos.
Plain CSS	–	Styling with CSS variables.	No framework needed; full control of the government look.
Web Speech API	Chrome	Speech-to-text (`SpeechRecognition`) and English speech (`speechSynthesis`).	Built into the browser; no extra service or key.
Web Audio API	Chrome	`AnalyserNode` measures mic and audio level for the orb.	Makes the orb react to real sound.
Playwright	1.63.0	End-to-end and visual tests.	Drives a real browser, reads DOM and localStorage, starts both servers.

Backend

Technology	Version	Role in eSiri	Why it was chosen
Python	3.12.3	Backend language.	Standard for AI integration.

Technology	Version	Role in eSiri	Why it was chosen
FastAPI	0.141.1	HTTP endpoints; request validation with pydantic models.	Asynchronous, so a slow OpenAI call does not block other requests.
uvicorn	0.54.0	ASGI server running FastAPI.	Standard FastAPI server.
openai (official SDK)	3.20.0	`AsyncOpenAI`: `chat.completions.create` for the agent, `audio.speech.create` for speech.	Official client; supports tool calling and speech.
pydantic	2.13.5	`StepRequest`, `ToolResult`, `TTSRequest` models.	Validates every request body.
python-dotenv	1.2.3	Loads `.env` from the project root.	Keeps the key out of code.

External services

Service	Used for	Called from
OpenAI Chat Completions (`gpt-4.1-mini`)	The agent's reasoning, via tool calling	`main.py` `step()`
OpenAI Audio Speech (`gpt-4o-mini-tts`, fallback `tts-1`)	Swahili speech; English too if `ESIRI_ENGLISH_TTS=openai`	`tts.py` `synthesize()`
Google speech service (inside Chrome)	Speech recognition	Browser `SpeechRecognition` in `stt.ts`
Google Fonts	Inter and Montserrat web fonts	`index.html`

4. How the agent knows what it can do

> **IN PLAIN WORDS**
>
> At every step the model receives four inputs: **rules** (the system prompt), **knowledge** (the app map), **abilities** (the tool definitions) and **eyes** (the screen snapshot). The snapshot can name every button precisely because each button in the mockup carries a fixed id tag.

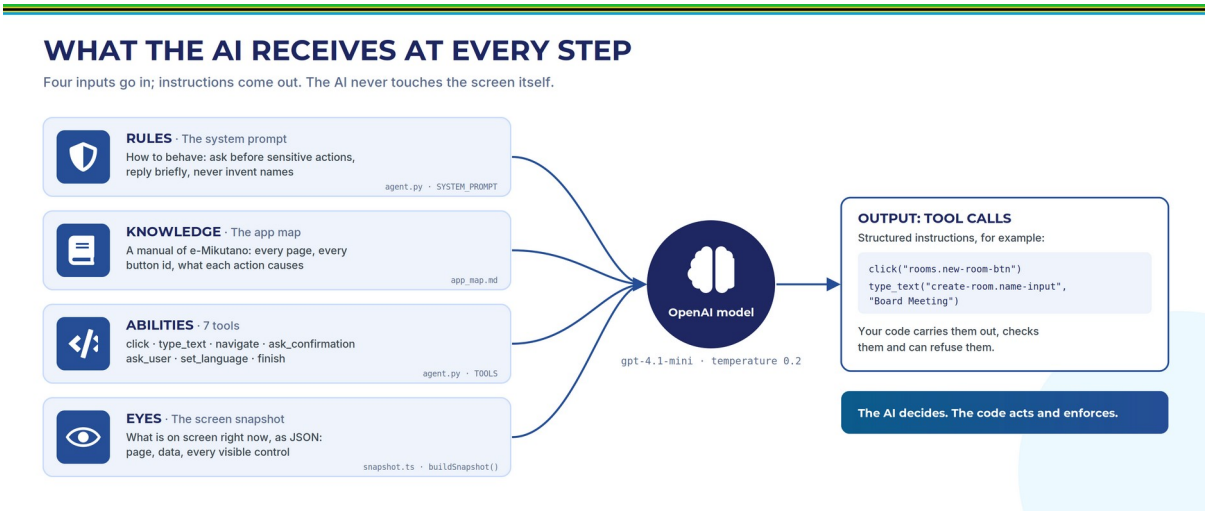


Figure 2. The four inputs sent on every model call, and the structured output.

4.1 Rules: the system prompt

`SYSTEM_PROMPT` in `backend/app/agent.py` is assembled by `build_system_prompt(language)`, which inserts the full app map and appends a line naming the current language. It has three sections:

- **How you work**: act only through tools; use only element ids from the latest snapshot, or ids that are certain to appear after a previous action in the same batch; prefer visible navigation; a click that opens another page must be the last call of its batch; read toggles before clicking; do only what was asked.
- **Safety and consent**: a closed list of sensitive actions that require `ask_confirmation` first; an explicit list of things that are not sensitive; after a decline, the next call must be `finish`; missing information means `ask_user`; never decide a room exists because a name looks similar or is a translation.
- **Language and style**: reply in the current language; never translate user data; replies are spoken, so one or two short sentences, no markdown, no ids; read access codes digit by digit.

Two representative rules, verbatim:

```
Before any sensitive action (creating a room, starting or joining a meeting, starting or
stopping a recording, leaving a meeting, removing someone's access, deleting a room or a
recording) you MUST call ask_confirmation with a short, specific summary.

If the user declines ({"confirmed": false}), do not perform that action and never ask
again:
your very next call must be finish with a short acknowledgement.
```

4.2 Knowledge: the app map

`backend/app/app_map.md` is the agent's knowledge base. It is loaded once when the backend starts (edits require a restart) and covers: how to read the snapshot; every page with its route, every element id and what it does, with sensitive ones marked; ids of data not yet visible, such as directory users (`neema-mwakalinga`) and sample documents (`meeting-agenda`); typical flows; and cause-and-effect rules, such as "mute on join makes the mic start muted" or "recording requires Allow recording".

4.3 Abilities: the seven tools

The tools are defined as JSON schemas in `agent.py TOOLS` and sent with every call. The model can only express intentions through them.

Tool	Parameters	What happens / returns
`click`	`element_id`, `reason`	Driver clicks the element with the visible cursor. Returns `{ok:true}` or an error code: `not_found`, `disabled`, `confirmation_required`.
`type_text`	`element_id`, `text`, `reason`	Clicks the field, clears it, types letter by letter. For the room-name field it also returns `duplicate_of` (deterministic duplicate check).
`navigate`	`page`, `room_id`	Goes directly to a page; only when no visible path exists. Entering a meeting consumes a confirmation credit; leaving a meeting this way is refused.
`ask_confirmation`	`summary`, `covers`	Shows a Yes/No card and speaks the summary. Returns `{confirmed:true|false}`. `covers` = number of sensitive clicks the approval allows (1–10).
`ask_user`	`question`	Asks one clarifying question and ends the turn; the answer arrives as the next user message in the same task.
`set_language`	`language` (`en`/`sw`)	Switches UI, recognition, speech and replies together.
`finish`	`message`	Speaks the final reply and ends the task.

4.4 Eyes, part 1: the tagging system

Every actionable element in the mockup is spread with `{...ez(id, label, opts)}` from `components/esiriProps.ts`, which produces these attributes:

Attribute	Meaning
`data-esiri-id`	Stable, semantic, language-independent id, e.g. `room.start-meeting`.
`data-esiri-label`	The visible label in the current language (changes with language; the id does not).
`data-esiri-sensitive="true"`	Clicking requires an approved confirmation.
`data-esiri-state`	`on`/`off` for toggles and meeting controls, `selected`/`unselected` for tabs, `open`/`closed` for menus.

Attribute	Meaning
`data-esiri-modal`	Marks an open dialog, so the snapshot shows only what is reachable.

Id convention: `<area>.<thing>[.<action>]`, lower-case, with real data ids embedded for list items, e.g. `room-card.{roomId}.start` or `access.shared.{userId}.remove`.

Sensitive elements (exact list): `landing.join-btn`, `room-card.{roomId}.start`, `create-room.submit`, `room.start-meeting`, `access.shared.{userId}.remove`, `delete-room.confirm`, `recording.{recId}.delete`, `meeting.record-toggle`, `meeting.leave`, `audit.clear-confirm`. In addition, `navigate` into a meeting requires a credit.

4.5 Eyes, part 2: the snapshot

`snapshot.ts` `buildSnapshot()` produces a compact JSON description before every model call. It contains the page and route, the language, any open dialog, visible notification texts, a `state` summary from the store (rooms with tokens; for the current room its settings, access codes, shared users, presentation files and recordings; in a meeting the recording, mic and camera state and the last 5 chat messages), and the list of visible tagged elements. When a dialog is open, only its elements are listed. This is a real snapshot captured with the Create Room dialog open:

```
{
  "page": "rooms", "route": "/rooms", "language": "en",
  "open_modal": "create-room", "toasts": [],
  "state": { "signed_in": true,
    "rooms": [ { "id": "rahma-mbuyus-room-r7k2", "name": "Rahma Mbuyu's Room",
                 "token": "RMR-7K2Q", "last_session": "September 25, 2026 11:11
AM" } ] },
  "elements": [
    { "id": "create-room.name-input", "role": "textbox", "label": "Room Name", "value":
"Board Meeting" },
    { "id": "create-room.close",      "role": "button",  "label": "Close" },
    { "id": "create-room.submit",     "role": "button",  "label": "Create Room",
"sensitive": true }
  ]
}
```

5. The agent loop in detail

> **IN PLAIN WORDS**
>
> The browser sends the request and a snapshot to the backend. The backend adds memory and rules, asks the model, and returns tool calls. The browser runs them one by one, then sends the results and a fresh snapshot back. This repeats until the model calls `finish`.

5.1 The browser–backend protocol

Everything goes through one endpoint, `POST /api/agent/step`, with two request shapes. A new user message:

```
{ "session_id": "f7f1506c-…", "language": "en", "input_mode": "text",
  "user_message": "Create a room called Board Meeting",
  "snapshot": { … } }
```

Results of the previous tool calls, with a fresh snapshot:

```
{ "session_id": "f7f1506c-…", "language": "en",
  "tool_results": [
    { "tool_call_id": "call_LhqK…", "result": { "ok": true } },
    { "tool_call_id": "call_S9i7…", "result": { "ok": true, "duplicate_of": null,
        "note": "No room with this exact name exists. Continue creating it." } } ],
  "snapshot": { … } }
```

The response is always a list of tool calls with arguments already parsed. If it contains none, the text is treated as the final message.

```
{ "tool_calls": [ { "id": "call_LhqK…", "name": "click",
    "arguments": { "element_id": "rooms.new-room-btn", "reason": "Open create new room
modal" } } ],
  "text": "" }
```

The `session_id` is a UUID generated by the browser (`crypto.randomUUID()`), kept in localStorage and replaced by "New conversation" or "Reset demo data".

5.2 How a batch of tool calls is executed

The model may return several tool calls at once (`parallel_tool_calls=True`). `controller.ts runLoop()` executes them **sequentially, in order**, under these rules:

1. If an earlier call failed, was declined or ended the turn, the rest get `{"skipped": true}` (OpenAI requires a result for every call id).
2. **Stale-prediction skip:** if the page changed during the batch and the next target does not appear within 1.5 s, the batch stops and the model is told to re-plan from the new snapshot.
3. **Execution cap:** at 30 executions in one task, the task fails.
4. A failed call stops the batch. Three consecutive failures of the same action fail the task.
5. `ask_confirmation` pauses the batch until answered; a decline skips the remaining calls.
6. `finish` ends the task; `ask_user` ends the turn and waits for the answer.

7. Otherwise, send all results with one fresh snapshot and loop.

5.3 Backend memory and prompt assembly

- **Session memory:** `SESSIONS` in `main.py` maps each session id to its list of OpenAI chat messages, in process memory. It persists across tasks, which is why follow-ups such as "now turn it off again" work. A per-session `asyncio.Lock` stops a cancelled request from interleaving with the next.
- **Snapshot pruning** (`agent.py render_messages()`): only the newest snapshot is sent in full; all older ones become the literal text `[old snapshot omitted]`. This keeps every call at about 4,200–4,500 tokens instead of growing with each step.
- **Repairing dangling tool calls** (`repair_dangling_tool_calls()`): OpenAI rejects a history in which a tool call has no result. Before each new user message, the backend inserts a placeholder result for any unanswered call (after `ask_user`, a cancel or a `finish`).
- **History trimming** (`trim_history()`): only the last 12 user turns are kept.
- **Duplicate-name hint:** each user message is followed by the exact list of existing room names, stating that any other name, including a translation, is a new room.
- The model is called with `temperature=0.2` (low randomness), a 60-second timeout and one retry.

5.4 Guards and error handling

Guard	Where	Value
Max tool executions per task	`controller.ts MAX_TOOL_EXECUTIONS`	30, then the task fails
Same action failing in a row	`controller.ts MAX_SAME_FAILURES`	3, then the task fails with the error
Re-asking after a decline	`execute() ask_confirmation`	Ends the task immediately
Waiting for predicted elements	`driver.ts find()`	150 ms for the first call of a batch, 1500 ms for later calls
Speech can never block	`tts.ts withTimeout()`	Estimated duration + 3 s (+6 s for OpenAI audio)
Recognition safety net	`stt.ts listenOnce()`	20 s

Backend errors are returned as `{error, code}` with codes such as `no_key` (400), `session_expired` (409), `model_not_found` (502), `auth` (401), `rate_limit` (429) and `network` (504). The browser speaks a localised error message and records the task as **Failed**.

6. Lifecycle of one real request

> **IN PLAIN WORDS**
>
> This is a real, instrumented run of "Create a room called Board Meeting". Four model calls, fifteen seconds. It is the best evidence that the AI is choosing the steps, not following a script.

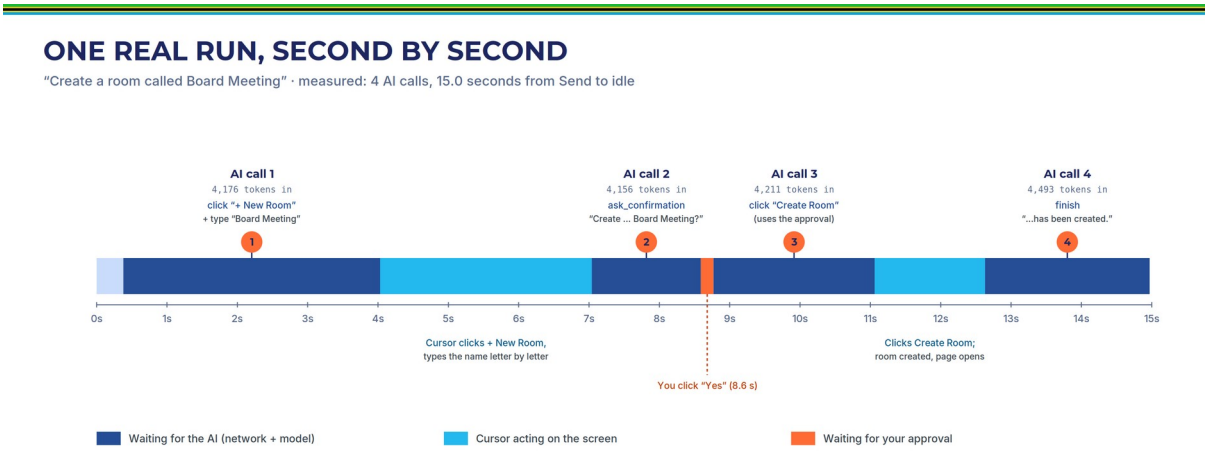


Figure 3. Status timeline of the captured run. Dark blue = waiting for the model; cyan = cursor acting; orange = waiting for approval.

Phase	Where	What happens
1. Activate	`EsiriRoot.tsx` → `controller.ts activate()`	Alt+S opens the panel (the page shrinks by 380 px so nothing is covered) and starts listening with `webkitSpeechRecognition`, `en-US`, interim results shown live.
2. Submit	`submit()` → `runLoop()`	Creates a `Task` (credits 0, abort controller), builds the snapshot, `POST /api/agent/step`.
3. Model call 1	`main.py step()`	Returns `click(rooms.new-room-btn)` and `type_text(create-room.name-input, "Board Meeting")`.
4. Act	`driver.ts click()`, `typeText()`	Cursor glides and clicks; the dialog opens; the name is typed at 35 ms per character; the duplicate check returns `duplicate_of: null`.
5. Model call 2	`step()`	Returns `ask_confirmation("Create a new room called Board Meeting")`. The card appears and the summary is spoken.
6. Approve	`answerConfirmation(true)`	User says or clicks Yes. `task.credits += 1`.
7. Model call 3	`step()` → `driver.ts`	Returns `click(create-room.submit)`. The element is sensitive; a credit exists, so the click proceeds and the credit is used. The room is created and its page opens.
8. Model call 4	`step()` → `finishTask()`	Returns `finish("The room called Board Meeting has been created successfully.")`. Audit record written; reply spoken; status idle.

What each model call cost

Call	Prompt tokens	Completion tokens	Model returned
1	4,176	71	`click` + `type_text`
2	4,156	20	`ask_confirmation`
3	4,211	28	`click(create-room.submit)`
4	4,493	22	`finish`

Most of each prompt is the 14,603-character system prompt, which is repeated on every call. Note how small the outputs are: the model returns short structured instructions, not text.

7. The hands: driver and ghost cursor

> **IN PLAIN WORDS**
>
> The driver is the only part that touches the screen. It finds the element, moves a visible cursor to it, and performs a genuine click, so the application reacts exactly as it would to a person. Typing needs a special technique because React controls the input fields.

7.1 The click sequence

`driver.ts click()`: find the element (visible, not inside the eSiri panel, not behind a dialog) → check it is enabled → **sensitive gate** (chapter 8) → scroll into view if needed → glide the cursor (500–700 ms, ease-in-out) and highlight the target → pause 140 ms → re-check the element → ripple and press animation → use a credit if sensitive → `el.click()` → settle (350 ms plus two animation frames, longer if the page changed) → return `{ok:true}`.

`el.click()` dispatches a real DOM click event, so React handlers, form submission and checkbox behaviour all work exactly as for a human.

7.2 Typing into React inputs: the native-setter technique

The mockup's inputs are **React controlled components**: the value shown comes from React state. Setting `input.value = "x"` directly does not work, because React overrides the element's `value` setter to track the last value it wrote. A direct assignment updates that tracker too, so when the input event fires React sees "no change", never calls `onChange`, and the next render restores the old value. The fix calls the **prototype's** original setter, which bypasses React's tracker:

```
const proto = el instanceof HTMLTextAreaElement ? HTMLTextAreaElement.prototype
                                                : HTMLInputElement.prototype;
const setter = Object.getOwnPropertyDescriptor(proto, 'value')?.set;
setter.call(el, value);                                  // bypass React's instance
setter
el.dispatchEvent(new Event('input', { bubbles: true })); // React listens for input
```

`typeText()` clears the field, then sets the growing string once per character every 35 ms, so the user sees it typed. It never sends an Enter key, so typing cannot submit a form, and it verifies the final value.

7.3 Navigate, settle and errors

- `navigate()` validates the page name, requires a known room id for room and meeting pages, consumes a credit to enter a meeting, refuses to leave a meeting (the Leave button must be used), and calls React Router through `nav.ts`.
- The driver never throws for interface problems. It returns `{ok:false, code, error}` (missing, disabled, behind a dialog, not editable), which the model reads and reacts to. Only cancellation throws, and the loop catches it silently.

8. Safety model

> **IN PLAIN WORDS**
>
> Important actions need the user's yes, and that is enforced by code in the browser, not left to the AI. A yes grants a small number of "credits" for sensitive clicks in that task only. The user can stop everything at any moment, and every task is recorded.

8.1 The confirmation gate

Enforced in `frontend/src/esiri/driver.ts`:

```
const sensitive = el.getAttribute('data-esiri-sensitive') === 'true';
if (sensitive && !ctx.hasCredit()) {
  return { ok: false, code: 'confirmation_required', error: 'confirmation_required: …' };
}
… cursor animation …
if (sensitive) ctx.useCredit();
el.click();
```

`ctx` is built per task by `controller.ts` `driverCtx(task)`: `hasCredit = () => task.credits > 0`. A refused click appears in the panel as "Needs confirmation" and returns `confirmation_required` to the model, which is instructed to ask and retry.

8.2 Credits and "covers"

Credits are added in exactly one place: `execute()` case `ask_confirmation`, after a yes, `task.credits += covers` (clamped to 1–10, default 1). Credits live on the `Task` object and disappear when it ends. For example, "stop recording and leave the meeting" is approved once with `covers: 2`.

8.3 Why in code and not only in the prompt

A prompt is a request to a probabilistic model; it can be ignored or misread. During the build, after a user declined deleting a room, the model asked for confirmation seven more times and then tried to click `delete-room.confirm`. **The gate refused the click and the room survived.** With the gate in the driver, a sensitive click without a yes in the same task is impossible, whatever the model outputs. A non-AI test proves this by calling the driver directly with zero credits.

8.4 Decline and cancellation

- **Decline** (No button, a spoken or typed "no" or "hapana", or two unclear voice answers): the confirmation is recorded as declined, no credits are granted, remaining batched calls are skipped, and the model is told not to ask again. If it asks again anyway, the task ends at once. Outcome: **Declined**.
- **Cancel** (Esc, the Stop button, clicking the orb while busy, or saying "stop", "cancel", "simama", "acha", "sitisha"): `cancel()` stops listening and speaking, hides the cursor, aborts the in-flight request through the task's `AbortController` (which also interrupts every driver wait), and records the task as **Cancelled**.

8.5 Limits of the gate

- It runs in the browser: strong against the model, but a person with developer tools could bypass it, just as they could click the button themselves. There is no server-side check.
- Credits are not bound to a specific element: an approval of N credits allows any N sensitive clicks in that task.
- It depends on correct tagging: an untagged consequential button would not be gated.

9. The roles of OpenAI and Google, in depth

> **IN PLAIN WORDS**
>
> OpenAI is the **brain** (it chooses tool calls) and the **Swahili voice**. Google, through Chrome, is the **ears** (speech-to-text). Neither one touches the screen or the data store. This chapter explains exactly what each receives and does.

9.1 What "tool calling" is

Normally a language model returns text. With **tool calling** (also called function calling), the application sends the model a list of functions it may use, each described by a JSON schema: a name, a description and typed parameters. The model can then reply with a structured request to call one or more of those functions, with arguments, instead of prose.

Crucially, **the model does not run anything**. It only produces the request. The application decides whether and how to execute it, and sends the outcome back as a "tool result" message. The model then decides the next step. This is what makes eSiri controllable: the model proposes, the code disposes.

In eSiri, the backend sends the seven tool schemas on every call. The model's reply contains `tool_calls` with ids; the browser executes them and returns one result per id. `parallel_tool_calls=True` allows several calls in one reply, which eSiri executes sequentially.

9.2 What OpenAI receives on every step

Data sent to OpenAI Chat Completions	Detail
System prompt	Rules + full app map + language line (14,603 characters).
Conversation history	User requests, the model's own earlier tool calls, and tool results; the last 12 user turns.
The latest screen snapshot	Room names and ids, **join tokens**, **viewer and moderator access codes**, names of people with access, file names, recording names; in meetings, the **last 5 chat messages**; current values of visible text fields; notification texts.
Tool definitions	The seven JSON schemas.

Not sent: the audit log, contents of files, the shared-notes text (only its length), and chat beyond the last five messages. This happens on every step, typically 3 to 5 times per task.

9.3 OpenAI text-to-speech

`backend/app/tts.py` `synthesize()` calls `audio.speech.create` with model `gpt-4o-mini-tts`, voice `coral`, MP3 output, and instructions asking for natural Kiswahili with a warm Tanzanian accent. On any error it retries once with `tts-1`. Results are cached in memory (up to 200 entries, keyed by a SHA-256 hash of language and text). The browser plays the MP3 through an `AnalyserNode` so the orb pulses with the real audio.

Swahili always uses OpenAI unless the computer has a local Swahili voice (the Linux build machine had none). English uses Chrome's own voice by default; setting `ESIRI_ENGLISH_TTS=openai` routes English through OpenAI too.

9.4 Google speech recognition, through Chrome

eSiri uses the browser's `SpeechRecognition` API with language `en-US` or `sw-TZ`. In Google Chrome this API streams microphone audio to **Google's speech service** and returns text, including interim partial results shown live. eSiri never handles the audio itself.

This is why voice needs **Google Chrome specifically**: Chromium has the same API but lacks access to Google's service, and fails with a `network` error on Linux. The separate microphone level stream used to animate the orb is analysed locally and never transmitted.

9.5 Other Google traffic

- English replies spoken with Chrome's "Google …" voices are very likely synthesised by Google (Chrome reports them as non-local). This was not verified in the project.
- Web fonts (Inter, Montserrat) load from Google Fonts at page load.

9.6 What stays local

- All application data (rooms, codes, chat, notes, recordings, audit) in the browser's localStorage.
- Backend session memory and `audit_log.jsonl`.
- The OpenAI key: only in `.env` and backend memory; never sent to the browser.

10. Speech layer

> **IN PLAIN WORDS**
>
> eSiri listens once per request, speaks its replies, understands spoken yes and no, and optionally listens for its name. It never listens while it is speaking or acting, so it cannot hear itself.

Speech-to-text (`stt.ts`)

- `listenOnce(lang)` creates a recognizer with interim results and `continuous = false`, resolves once with text or an error, and has a 20-second safety timer. Only one recognizer runs at a time.
- Errors are mapped to clear messages: `no-speech` (quietly back to idle), blocked microphone, `network` ("needs Google Chrome and internet; you can type instead"), and unsupported browser (typing is focused instead).

Text-to-speech (`tts.ts` `speak(text, lang)`)

- Strips markdown, stops any older speech, then routes: English to the best Chrome voice (preferring "Google" voices) or OpenAI; Swahili to a local `sw` voice if present, otherwise OpenAI.
- Browser speech is split into chunks of at most 180 characters at sentence boundaries, because Chrome cuts long utterances.
- Every `speak()` has a timeout (estimated duration + 3 s, +6 s for OpenAI audio) and never rejects, so speech can never freeze the agent loop.

Spoken yes and no (`util.ts` `parseYesNo()`)

Whole-word matching. Yes: yes, yeah, yep, sure, ok, okay, go ahead, do it, proceed, confirm, ndiyo, ndio, sawa, endelea, haya, thibitisha. No: no, nope, cancel, don't, stop, hapana, acha, usifanye, ghairi, sitisha. Unclear answers get one "Please say yes or no"; a second unclear answer counts as no. The same parser accepts typed answers.

Wake word and echo avoidance

- Wake mode (off by default) runs continuous recognition only while idle, restarting after each end, and matches `/\b(e[\s-]?siri|hey siri|hi siri|a siri|easy ri|siri)\b/i`, because recognisers spell "eSiri" in many ways. Text after the wake word is used directly as the command.
- Listening starts only from activation, or after a confirmation or question has finished being spoken; wake mode pauses whenever eSiri is busy.

11. Bilingual support

> **IN PLAIN WORDS**
>
> One language setting drives everything: screen text, the recognition language, the voice and the AI's replies. User data such as room names is never translated.

- **Dictionaries:** `i18n/en.ts` (261 keys) and `i18n/sw.ts`, typed so that TypeScript refuses to compile if Swahili is missing or has extra keys.
- **One setting:** `useApp.language` (persisted). Three entry points change it: the landing toggle, the panel chip, and the agent's `set_language` tool.
- **Propagation:** every component re-renders labels through `useT()` (ids never change); recognition uses `en-US` or `sw-TZ`; speech routing picks the voice; every backend request carries `language`, which appears in the user message and in the system prompt's last line.
- **Model-facing text stays in English:** the app map, tool notes and driver errors are English even in Swahili sessions; they are never shown to the user.

12. State and persistence

> **IN PLAIN WORDS**
>
> The mockup's data lives in the browser (localStorage). The conversation shown in the panel lives in memory. The backend keeps conversation memory and an audit file. Nothing is in a database.

Store	Where	Contents	Persistence
`useApp`	`store/appStore.ts` (zustand + persist)	language, signedIn, sessionId, rooms (settings, codes, shared users, files), recordings, meetings (chat, notes), audit (max 500), wakeMode	localStorage key `emikutano-store`
`useApp.live`, toasts	same	Current meeting state; notifications	Not persisted
`useEsiri`	`esiri/esiriStore.ts`	panel, status, messages, interim text, health	Not persisted (a reload clears the panel)
Controller variables	`controller.ts`	current task, pending confirmation	Not persisted
Backend memory	`main.py`	`SESSIONS`, `LOCKS`, TTS cache	Process memory; lost on restart
Audit file	`backend/audit_log.jsonl`	One JSON line per task	Disk; git-ignored

Each **audit record** contains id, timestamp, user, language, input mode, request, steps (tool, target label, text, ok, error), confirmations (summary, result), outcome (Completed / Declined / Cancelled / Failed), final message and duration. The Audit page reads the browser copy and can export it to CSV.

13. Testing

> **IN PLAIN WORDS**
>
> 27 automated tests drive a real browser. They check the final result rather than exact steps, because the AI's path varies. The safety gate is tested without the AI at all. Voice is not covered automatically.

Suite	Tests	What it proves
`ui.spec.ts`	14 (no AI)	Every screen by hand, sign-in gate, language toggle, token join, meeting features, audit export, Alt+S panel, the **sensitive-click gate without the AI**, snapshot content, TTS and audit endpoints.
`agent.spec.ts`	8 (AI)	Create room; settings + viewer code; share access; start meeting + chat with mic muted; record, stop and leave; Swahili room "Kikao cha Bodi"; **decline a deletion** (room survives, exactly one confirmation); **Esc cancel** mid-action.
`demo.spec.ts`	1 (AI)	The full README demo as one conversation (13 requests, including memory follow-ups and Swahili); at least 12 Completed audit records.
`screenshots.spec.ts`	4	Every page in EN and SW at 1366×768 and 1920×1080 with the panel open; no overflow.

Configuration: Playwright Chromium, 120 s per test, one retry, serial workers, fresh storage per test, `?mute=1` (no microphone or speech). All passed in the final build run; during documentation the demo test failed once on the first attempt (the Swahili duplicate issue, chapter 15) and passed on retry.

Not covered automatically: real microphone and speakers, wake mode, voice confirmations, OpenAI English speech, and models other than `gpt-4.1-mini`.

```
cd frontend
npm run build          # type-check + production build
npx playwright test    # all 27 tests
npm run test:e2e:nollm # 18 tests without the AI
npm run test:e2e:llm   # 9 AI tests (needs the key)
```

14. Engineering decisions and bugs fixed

> **IN PLAIN WORDS**
>
> Several behaviours were moved from "asking the AI nicely" into deterministic code, because prompt rules alone were not reliable. That is the main engineering lesson of the build.

Bugs found and fixed

Symptom	Cause	Fix
After a declined deletion, the model asked 7 more times, then tried to delete (blocked by the gate).	The model did not treat "no" as final.	Decline result carries an explicit note; prompt says next call must be `finish`; code ends the task if it asks again; test asserts exactly one confirmation.
"Kikao cha Bodi" answered with "that room already exists".	The model equated the Swahili name with its English translation "Board Meeting".	Deterministic duplicate check (`duplicate_of`) on the name field; exact existing names added to each request. Mitigated, not eliminated.
Model clicked notification close buttons and guessed ids on pages not yet loaded.	Close button visible in snapshot; predictions after page changes.	`toast.close` removed from the snapshot; stale predictions skipped silently.
Model asked for confirmation on settings changes.	Over-cautious reading of rules.	Prompt lists explicitly what is not sensitive.
Microphone stream could stay open.	Race in `startMicLevel()`.	Generation counter.
Layout overlaps (title wrap, toasts over orb, webcam over slide).	CSS layout.	Container-query units, repositioned elements.

Decisions made during the build

- Token join (`landing.join-btn`) and clearing the audit log are treated as sensitive.
- Inner pages require sign-in; the meeting route does not, so token joins work.
- The snapshot is sent once per step (attached to the last tool result), not in every result.
- The backend repairs unanswered tool calls, and uses a per-session lock.
- `navigate` into a meeting consumes a credit, and `navigate` out of a meeting is refused, so `navigate` cannot bypass the gate.
- A confirmation may be the first call of a batch, followed by the actions; a decline skips the rest.
- Typed yes/no is accepted while a confirmation card is open, so typing is a full fallback for voice.
- Leaving a meeting while recording saves the recording.

15. Known limitations

> **IN PLAIN WORDS**
>
> Knowing these, and saying them first, is part of owning the project.

Approach

- **The mockup was built for the agent.** Every element has a stable id, label, sensitivity flag and state. A real third-party system will not; it needs an equivalent layer (chapter 16).
- The "system" is a front-end mockup: no real video, recording, sharing or server-side data.
- Single hard-coded user; no authentication on the backend endpoints.

Safety

- The gate runs in the browser; no server-side check. Credits are not bound to the approved element. An untagged consequential button would not be gated.
- `window.__esiriTest` (a test hook with direct access to the driver and stores) is present in all builds.
- The audit is not tamper-evident: `/api/audit` accepts any JSON, and the browser copy can be cleared.

Model behaviour

- Output varies between runs; several behaviours had to be fixed in code. Only `gpt-4.1-mini` was tested.
- Swahili names that translate an existing room can still trigger a wrong "already exists" question (once in 6 demo runs), because the model can ask before the deterministic check runs.
- The model later sees its own `finish` call marked as "skipped" in the history; the effect is unknown but harmless in tests.

Performance, dependencies and hygiene

- About 15 s for a simple task; about 4.2–4.5k prompt tokens per call. No explicit prompt caching configured.
- Backend sessions are never evicted; `/api/agent/reset` is never called by the UI.
- Needs internet (OpenAI, Google) and Google Chrome for voice. Swahili recognition quality not measured. The wake-word pattern includes bare "siri", so false activations are possible.
- No unit tests. Some unused exports remain: `resetSession()`, `hasPendingTask()`, `trFor()`, `isWakeActive()`, `getRoom()`.

16. Reusing the pattern in other government systems

> **IN PLAIN WORDS**
>
> The reusable core is the loop: **snapshot → model with tools → validated actions → confirmation gate → audit**. What changes per system is how you see its screen, how you act on it, and the manual you give the model.

Reusable as-is	Specific to each system
The agent loop and batch rules (`controller.ts runLoop()`)	How elements are identified (the tags, or an equivalent)
Session memory, snapshot pruning, history repair (`agent.py`)	The app map (`app_map.md`)
The confirmation gate, credits, decline and cancel logic	The snapshot `state` summary
Guards, audit record, speech layer, bilingual pattern	The list of sensitive actions and the prompt wording

Steps

1. **Decide how to act on the system.** A web UI you control: add `data-esiri-*` tags, as done here. A web UI you cannot change: run the "hands" server-side with Playwright, and build element ids from stable selectors and accessibility roles in a curated mapping, where sensitive actions are also marked. A system with an API: make the tools API calls (e.g. `create_room(name)`) and keep confirmation and credits in the executor.
2. **Write the snapshot builder** for that system: page, open dialogs, visible actionable elements with ids, labels and states, plus a compact domain state. Keep it small and prune old snapshots.
3. **Write the app map** (pages, ids, cause and effect, sensitive actions) and adjust the system prompt.
4. **Keep enforcement in the executor:** the credit gate, decline ends the task, cancel aborts everything, step and failure guards, audit every task.
5. **Add deterministic checks** for things the model gets wrong, like `duplicate_of` here.
6. **Test on final state** with an AI suite, plus non-AI tests for the gate.

17. Resuming work on the project

> **IN PLAIN WORDS**
>
> Open the project in Claude Code: it reads `CLAUDE.md` automatically, and `docs/CODEMAP.md` holds the detail. Start with `./start.sh`, and use the table below to find where to change things.

Run it

```
cp .env.example .env     # add OPENAI_API_KEY=sk-...
./start.sh               # installs on first run, then starts :8000 and :5173
# open http://localhost:5173 in Google Chrome (not Chromium), allow the microphone
```

Configuration (`.env`)

Variable	Default	Controls
`OPENAI_API_KEY`	none	Required for the agent and speech; without it the panel shows a banner.
`OPENAI_MODEL`	`gpt-4.1-mini`	Agent model.
`OPENAI_TTS_MODEL`	`gpt-4o-mini-tts`	Speech model (falls back to `tts-1`).
`OPENAI_TTS_VOICE`	`coral`	OpenAI voice.
`ESIRI_ENGLISH_TTS`	`browser`	`browser` or `openai` for English speech.

URL parameters: `?mute=1` disables speech and microphone (tests); `?token=XXXX` pre-fills the join token.

Where to change what

If you want to…	Change
Change how the agent behaves	`backend/app/agent.py` `SYSTEM_PROMPT` (restart backend)
Teach it about a page or button	`backend/app/app_map.md` (restart backend)
Add a new kind of action (tool)	`agent.py` `TOOLS` + `controller.ts` `execute()` + i18n step text + app map
Make a button require confirmation	Add `{ sensitive: true }` in its `ez()` call; list it in the prompt's safety section
Change what the model sees	`frontend/src/esiri/snapshot.ts` `buildSnapshot()`
Change click/typing behaviour	`frontend/src/esiri/driver.ts`, `cursor.ts`
Change loop limits or batching	`frontend/src/esiri/controller.ts` (`MAX_TOOL_EXECUTIONS`, `MAX_SAME_FAILURES`)
Change voices or languages	`esiri/tts.ts`, `esiri/stt.ts`, `backend/app/tts.py`, `i18n/en.ts` + `sw.ts`

If you want to…	Change
Add a mockup screen	New page in `pages/`, route in `App.tsx`, tags via `ez()`, `nav.ts` `pageFromPath()`, app map, tests

Rules not to break

- Keep the confirmation gate in the driver; never move enforcement into the prompt only.
- Never commit `.env`. Every new string goes into both `en.ts` and `sw.ts`. Follow the id convention `<area>.<thing>[.<action>]`.
- Run `npm run build` and the test suite after every change.

18. Glossary

Term	Meaning in this codebase
Agent loop	`controller.ts runLoop()`: snapshot → model → execute tool calls → results → repeat until `finish`.
Task / turn	A task is one request from submit to finish, fail, decline or cancel (one audit record). A turn is one user message and the model calls it triggers; `ask_user` spans turns.
Tool / tool call / tool result	A function the model may call; one invocation with id and JSON arguments; the JSON the browser returns for that id.
Batch	All tool calls returned in one model response, executed in order.
Snapshot / pruning	JSON description of the screen sent to the model; replacing older snapshots with "[old snapshot omitted]".
Session	Server-side conversation memory, keyed by a UUID stored in the browser.
App map / system prompt	The agent's knowledge base; its standing rules (prompt + app map + language line).
Element id	The value of `data-esiri-id`, e.g. `room.start-meeting`.
Sensitive action / gate / credit	A click on an element tagged sensitive; the driver check refusing it without credits; one allowed sensitive click granted by an approved confirmation.
Outcome	Completed, Declined, Cancelled or Failed.
Driver / ghost cursor / settle	The code that acts on the page; the animated pointer; the wait after an action for React and routing to update.
Native setter / controlled component	The prototype value setter used for typing; a React input whose value lives in React state.
Stale prediction	A batched call whose target is not on the new page.
Tool calling / parallel tool calls	Model feature returning structured function requests; several per response.
Temperature	A setting for how random the model's choices are; 0.2 is low (more consistent).
Token	The unit a model reads and writes; about three-quarters of an English word.
STT / TTS	Speech-to-text / text-to-speech.
Vite proxy	Dev-server forwarding of `/api/*` to the backend.
FastAPI / uvicorn / pydantic	Python web framework / its server / request validation models.
zustand / persist	State library / its localStorage middleware.
Playwright / @llm tag	Browser test tool / marks tests that need the OpenAI key.